[INSTITO POLITÉCNICO SUPERIOR]

[Departamento de Informática]

Analista Universitario en Sistemas

Taller de programación II

Trabajo Práctico

Implementación del Algoritmo de Dijkstra para Enrutamiento en una Red Simulada

---

Tema: Algoritmo de Dijkstra aplicado a redes

Modalidad: Individual o en parejas

Lenguaje: C (estándar C11 o superior)

Fecha: 25 de noviembre

[2026 / Segundo Cuatrimestre]

Índice

1. Objetivos **3**

1.1. Objetivo General . . . 3

1.2. Objetivos Específicos . . . 3

2. Marco Teórico: ¿Cómo funciona el Algoritmo de Dijkstra? **3**

2.1. Contexto en redes . . . 3

2.2. Idea central . . . 3

2.3. Pasos del algoritmo . . . 4

2.4. Ejemplo paso a paso . . . 4

2.5. Cálculo del *next hop* (tabla de enrutamiento) . . . 5

2.6. Restricciones y propiedades . . . 5

2.7. Relación con OSPF y enrutamiento real . . . 5

2.7.1. ¿Qué es OSPF? . . . 6

2.7.2. Funcionamiento básico: LSAs, flooding y LSDB . . . 6

2.7.3. El costo en OSPF . . . 6

2.7.4. Jerarquía de áreas y tipos de routers . . . 6

2.7.5. Comparación con RIP . . . 7

2.7.6. Qué simula y qué no simula nuestro trabajo práctico . . . 7

3. Descripción del Problema **8**

4. Estructura del Proyecto **8**

5. Requerimientos Técnicos **8**

5.1. Representación de la Red . . . 8

5.2. Estructuras de Datos Sugeridas . . . 9

5.3. Funciones Mínimas Requeridas . . . 9

5.4. Requisitos de Implementación del Algoritmo . . . 9

6. Interfaz del Programa (Menú) **10**

6.1. Topología de Prueba (ejemplo) . . . 10

7. Formato del Archivo de Topología **10**

8. Criterios de Evaluación **11**

9. Casos de Prueba Obligatorios **11**

10.Entregables **12**

11.Pistas y Sugerencias **12**

12.Cronograma Sugerido **13**

13.Referencias Sugeridas **13**

14.Material Adjunto **13**

A. Anexo 1: grafo.h **15**

B. Anexo 2: dijkstra.h **15**

C. Anexo 3: enrutamiento.h **15**

D. Anexo 4: Test automatizados **16**

E. Anexo 5: main.c **20**

F. Anexo 6: Makefile de ejemplo **22**

G. Anexo 7: Ejemplo de bash de ejecución **22**

1. Objetivos

1.1. Objetivo General

Implementar el algoritmo de Dijkstra en lenguaje C para calcular las rutas de menor costo en una red de computadoras simulada, emulando el comportamiento de un protocolo de estado de enlace (como OSPF).

1.2. Objetivos Específicos

1. Comprender el funcionamiento del algoritmo de Dijkstra y su rol en el enrutamiento de redes.
2. Representar una topología de red mediante una estructura de datos adecuada (matriz o lista de adyacencia).
3. Implementar el algoritmo de Dijkstra para obtener el árbol de rutas más cortas desde un router origen.
4. Construir la tabla de enrutamiento (*next hop*) a partir de los resultados.
5. Simular el envío de paquetes entre nodos siguiendo las rutas calculadas.
6. Aplicar buenas prácticas de programación en C: modularización, manejo de memoria, validación de entradas.

2. Marco Teórico: ¿Cómo funciona el Algoritmo de Dijkstra?

2.1. Contexto en redes

En una red de computadoras, cada router necesita saber **por dónde enviar un paquete** para que llegue a su destino con el menor costo posible (menor retardo, mayor ancho de banda, menos saltos, etc.). Los protocolos de **estado de enlace** como OSPF resuelven este problema en dos fases:

1. **Descubrimiento de la topología:** cada router intercambia información con sus vecinos hasta que todos tienen un "mapa" completo de la red (la *Link State Database*).
2. **Cálculo de rutas:** cada router ejecuta el **algoritmo de Dijkstra** sobre ese mapa para obtener el árbol de caminos más cortos desde sí mismo hacia todos los destinos.

El algoritmo de Dijkstra fue propuesto por Edsger W. Dijkstra en 1956 y resuelve el problema del **camino más corto desde una fuente única** (*single-source shortest path*) en grafos con pesos **no negativos**. Su complejidad es $O(N^2)$ con matriz de adyacencia y $O((N + E) \log N)$ con cola de prioridad (*min-heap*).

2.2. Idea central

Dijkstra es un algoritmo **voraz** (*greedy*) que construye la solución de forma incremental. Mantiene tres estructuras:

- `distancia[v]`: el mejor costo conocido desde el origen hasta v.
- `predecesor[v]`: el nodo anterior en el mejor camino encontrado hacia v.
- `visitado[v]`: indica si v ya fue "cerrado" (su distancia es definitiva).

La **invariante** del algoritmo es: *cuando un nodo se marca como visitado, su distancia ya es la mínima posible.* Esto se garantiza porque los pesos son no negativos.

2.3. Pasos del algoritmo

Dado un grafo $G = (V, E)$ con pesos $w(u, v) \geq 0$ y un nodo origen s:

1. **Inicialización:**
   - `distancia`$[s] = 0$ y `distancia`$[v] = \infty$ para todo $v \neq s$.
   - `predecesor`$[v] = -1$ para todo v.
   - `visitado`$[v]$ = falso para todo v.
2. **Iteración principal** (repetir $|V|$ veces o hasta que no queden nodos alcanzables):
   - **Seleccionar** el nodo u no visitado con la menor `distancia`$[u]$.
   - Si no hay ningún nodo alcanzable, terminar.
   - Marcar u como **visitado** (su distancia ya es definitiva).
   - **Relajar** todas las aristas salientes de u: para cada vecino v de u no visitado, si

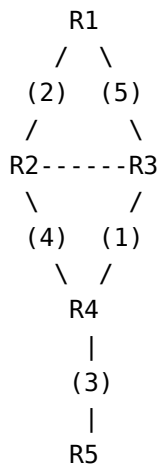
     $$\texttt{distancia}[u] + w(u, v) < \texttt{distancia}[v]$$

     entonces actualizar:

     $$\texttt{distancia}[v] = \texttt{distancia}[u] + w(u, v), \qquad \texttt{predecesor}[v] = u.$$
3. **Reconstrucción del camino:** para obtener la ruta hacia un destino d, seguir los `predecesor[]` desde d hasta s y luego invertir la secuencia.

2.4. Ejemplo paso a paso

Red del enunciado:



Ejecutando Dijkstra desde **R1**:

Iter	**Nodo elegido**	`dist[R2]`	`dist[R3]`	`dist[R4]`	`dist[R5]`
0	(inicio)	∞	∞	∞	∞
1	**R1** (0)	**2** (vía R1)	**5** (vía R1)	∞	∞
2	**R2** (2)	—	**3** (vía R2)	**6** (vía R2)	∞
3	**R3** (3)	—	—	**4** (vía R3)	∞
4	**R4** (4)	—	—	—	**7** (vía R4)
5	**R5** (7)	—	—	—	—

Resultado: `distancia` $= [0, 2, 3, 4, 7]$, `predecesor` $= [-1, 0, 1, 2, 3]$.

Reconstrucción del camino R1→R5: $R5 \leftarrow R4 \leftarrow R3 \leftarrow R2 \leftarrow R1$, invertido: **R1** $\rightarrow$ **R2** $\rightarrow$ **R3** $\rightarrow$ **R4** $\rightarrow$ **R5**, costo total 7.

2.5. Cálculo del *next hop* (tabla de enrutamiento)

En un router real no alcanza con saber el camino completo: se necesita saber **a qué vecino directo entregar el paquete** (el *siguiente salto*). Se obtiene retrocediendo por los predecesores desde el destino hasta encontrar el nodo cuyo predecesor es el origen. Ese nodo es el *next hop*.

Ejemplo: para R1→R5, el camino es R1 $\rightarrow$ R2 $\rightarrow$ R3 $\rightarrow$ R4 $\rightarrow$ R5. El nodo cuyo predecesor es R1 es **R2**, por lo tanto el *next hop* es R2. La tabla de R1 queda:

Destino	**Costo**	**Next Hop**
R2	2	R2
R3	3	R2
R4	4	R2
R5	7	R2

2.6. Restricciones y propiedades

- **Pesos no negativos:** el algoritmo no funciona con aristas de peso negativo (para eso existe Bellman-Ford).
- **Grafo no dirigido:** en redes, los enlaces suelen ser bidireccionales, por lo que $w(u, v) = w(v, u)$.
- **Sin bucles:** como cada nodo se visita una sola vez, las rutas generadas son simples (no contienen ciclos).
- **Complejidad:** $O(N^2)$ con matriz y búsqueda lineal del mínimo; $O((N + E) \log N)$ con *min-heap*.

2.7. Relación con OSPF y enrutamiento real

Hasta aquí hemos visto Dijkstra como un algoritmo abstracto sobre un grafo. En el mundo real, el protocolo que utiliza este algoritmo de forma masiva en redes IP es **OSPF** (*Open Shortest Path First*). Comprender cómo funciona OSPF permite entender por qué el trabajo práctico se centra precisamente en Dijkstra y qué simplificaciones introduce nuestra simulación respecto de un protocolo de producción.

2.7.1. ¿Qué es OSPF?

OSPF es un protocolo de enrutamiento **de estado de enlace** (*link-state*) para redes IP, definido en el **RFC 2328** para OSPFv2 (IPv4) [1] y en el RFC 5340 para OSPFv3 (IPv6) [2]. Es un estándar abierto gestionado por la IETF y, junto con IS-IS, es uno de los protocolos IGP (*Interior Gateway Protocol*) más utilizados en redes empresariales y de proveedores de servicios [6].

A diferencia de los protocolos de **vector de distancia** (como RIP), donde cada router intercambia con sus vecinos una lista de rutas ya calculadas, en OSPF **cada router describe únicamente el estado de sus enlaces directos** y esa información se difunde a todos los demás routers del dominio. De este modo, todos los routers construyen una visión completa e idéntica de la topología y calculan sus rutas de forma independiente mediante Dijkstra.

2.7.2. Funcionamiento básico: LSAs, flooding y LSDB

El funcionamiento de OSPF puede resumirse en cuatro etapas:

1. **Descubrimiento de vecinos (Hello Protocol).** Cada router envía periódicamente mensajes *Hello* por multicast (224.0.0.5) para descubrir otros routers en sus enlaces directos y verificar que la comunicación bidireccional sigue activa [1].
2. **Intercambio de LSAs.** Una vez establecidas las adyacencias, cada router genera **LSAs** (*Link State Advertisements*): mensajes que describen sus enlaces, sus vecinos y el **costo** asociado a cada uno [1].
3. **Flooding y construcción de la LSDB.** Los LSAs se propagan por *flooding* a todos los routers del área. Cada router almacena los LSAs recibidos en su **LSDB** (*Link State Database*), que es una representación completa de la topología del área. Cuando un enlace cambia de estado, solo es necesario difundir el LSA correspondiente; los demás routers actualizan su LSDB y recalculan sus rutas [4].
4. **Cálculo de rutas con SPF.** Con la LSDB completa, cada router ejecuta el algoritmo **SPF** (*Shortest Path First*), que no es otra cosa que el algoritmo de Dijkstra, tomando como origen (raíz del árbol) a sí mismo [5]. El resultado es un árbol de caminos mínimos hacia todas las redes y routers alcanzables, a partir del cual se construye la **tabla de enrutamiento** final [1].

2.7.3. El costo en OSPF

La métrica que utiliza OSPF es el **costo** (*cost*). Por defecto, el costo de una interfaz se calcula como:

$$\text{costo} = \frac{10^8}{\text{ancho de banda en bps}}.$$

Así, un enlace de 100 Mbps tiene costo 1, uno de 10 Mbps tiene costo 10, y así sucesivamente. Cuanto menor es el costo, más preferible resulta el enlace para el algoritmo SPF. Esto contrasta con RIP, cuya métrica es simplemente el **número de saltos** y no distingue entre un enlace lento y uno rápido [3].

2.7.4. Jerarquía de áreas y tipos de routers

Uno de los mecanismos que hace escalable a OSPF es la división del dominio en **áreas**. Un área es un conjunto de routers que comparten la misma LSDB. Todas las áreas deben conectarse a

través de un área central llamada **área 0** o *backbone* [1].

Esta jerarquía da lugar a distintos tipos de routers según su posición en la topología:

- **IR** (*Internal Router*): router con todas sus interfaces dentro de una misma área.
- **ABR** (*Area Border Router*): router conectado a dos o más áreas; genera LSAs de resumen para intercambiar información entre ellas.
- **BR** (*Backbone Router*): router con al menos una interfaz en el área 0.
- **ASBR** (*Autonomous System Boundary Router*): router que conecta el dominio OSPF con otro sistema autónomo o con rutas externas [1].

Además, en redes de acceso múltiple (como Ethernet) OSPF elige un **DR** (*Designated Router*) y un **BDR** (*Backup Designated Router*) para centralizar el intercambio de LSAs y reducir el tráfico de flooding [1].

2.7.5. Comparación con RIP

La siguiente tabla resume las diferencias más relevantes entre OSPF y RIP, el protocolo de vector de distancia clásico:

Aspecto	**RIP**	**OSPF**
Tipo de protocolo	Vector de distancia	Estado de enlace
Métrica	Número de saltos	Costo (basado en ancho de banda)
Convergencia	Lenta	Rápida
Escalabilidad	Baja (máx. 15 saltos)	Alta (sin límite de saltos)
Uso de recursos	Bajos	Altos
Complejidad de configuración	Baja	Alta
Estándar	Abierto	Abierto (IETF)

Fuente: elaboración propia a partir de [6, 7].

2.7.6. Qué simula y qué no simula nuestro trabajo práctico

El simulador desarrollado en este TP captura el **núcleo algorítmico** de OSPF: la existencia de una topología conocida (nuestra matriz de adyacencia), la asignación de costos a los enlaces y la ejecución de Dijkstra desde un origen para obtener distancias mínimas y una tabla de *next hop*.

Sin embargo, **no implementa** los mecanismos propios del protocolo que están fuera del alcance de un curso introductorio de programación en C:

- No hay mensajes *Hello* ni formación de adyacencias.
- No hay LSAs ni flooding: la topología se carga de forma estática desde un archivo o se ingresa manualmente.
- No hay jerarquía de áreas ni tipos de routers (IR, ABR, ASBR).

- No hay temporizadores, números de secuencia ni envejecimiento de LSAs.

Aun así, la lógica de `dijkstra()` y `construirTablaRutas()` es esencialmente la misma que ejecuta un router OSPF real al calcular su árbol SPF. Entender esta correspondencia es uno de los objetivos conceptuales del trabajo práctico: ver cómo un algoritmo de grafos se traduce en la decisión concreta de «por qué interfaz enviar este paquete».

3. Descripción del Problema

Se debe simular una red de computadoras compuesta por **N routers** interconectados por enlaces con distintos costos (que representan métricas como retardo o ancho de banda). Cada router debe poder:

1. Calcular el camino de menor costo hacia todos los demás routers usando Dijkstra.
2. Generar su **tabla de enrutamiento**, indicando para cada destino: costo total y siguiente salto (*next hop*).
3. Permitir simular el envío de un "paquete" desde un router origen a un destino, mostrando la ruta completa recorrida.

4. Estructura del Proyecto

El programa debe organizarse en los siguientes módulos:

```
trabajo_practico/
├── main.c           // Menú principal y control del programa
├── grafo.c          // Funciones de gestión del grafo
├── grafo.h          // Declaraciones de estructuras y prototipos
├── dijkstra.c       // Implementación del algoritmo
├── dijkstra.h
├── enrutamiento.c   // Tabla de rutas y simulación de envío
├── enrutamiento.h
├── tests.c          // Batería de pruebas automáticas
├── tests.h
├── Makefile
└── topologia.txt    // Ejemplo de topología de entrada
```

5. Requerimientos Técnicos

5.1. Representación de la Red

Se sugiere usar una **matriz de adyacencia** de tamaño $N \times N$, donde:

- `matriz[i][j] = costo` si existe un enlace entre el router i y j.
- `matriz[i][j] = 0` si $i = j$.
- `matriz[i][j] = INF` si no hay enlace directo.

Opcionalmente (para puntaje extra), implementar también con **lista de adyacencia**.

5.2. Estructuras de Datos Sugeridas

```c
#define MAX_ROUTERS 20
#define INF 999999
#define MAX_NOMBRE  32

/* Red simulada: matriz de adyacencia + nombres de routers.
 * Convención: matriz[i][j] == INF significa "sin enlace directo".
 *             matriz[i][i] == 0 siempre. */

typedef struct {
    int  numRouters;
    int  matriz[MAX_ROUTERS][MAX_ROUTERS];
    char nombres[MAX_ROUTERS][MAX_NOMBRE];
} Red;

typedef struct {
    int distancia[MAX_ROUTERS];
    int predecesor[MAX_ROUTERS];
    int visitado[MAX_ROUTERS];
} ResultadoDijkstra;

typedef struct {
    int destino;
    int costo;
    int siguienteSalto; /* -1 si inalcanzable */
} EntradaRuta;
```

5.3. Funciones Mínimas Requeridas

Función	Descripción
`inicializarRed()`	Inicializa la red con N routers sin enlaces.
`setNombreRouter()`	Asigna un nombre legible a un router.
`agregarEnlace()`	Agrega un enlace bidireccional con costo.
`eliminarEnlace()`	Elimina un enlace existente.
`cargarTopologiaDesdeArchivo()`	Carga la topología desde un archivo de texto.
`cargarTopologiaEjemplo()`	Carga una topología hardcodeada de prueba.
`mostrarTopologia()`	Imprime la matriz de adyacencia.
`dijkstra()`	Ejecuta el algoritmo.
`construirTablaRutas()`	Genera la tabla con *next hop* por destino.
`mostrarTablaRutas()`	Imprime la tabla formateada.
`simularEnvioPaquete()`	Muestra la ruta completa de un paquete.
`liberarRed()`	Libera recursos.

5.4. Requisitos de Implementación del Algoritmo

- Usar un arreglo `distancia[]` inicializado en `INF` salvo el origen (`0`).
- Usar `visitado[]` para marcar nodos ya procesados.
- En cada iteración, seleccionar el nodo no visitado con menor distancia (búsqueda lineal es aceptable; implementar con **min-heap** otorga puntaje extra).
- Relajar aristas: si `distancia`$[u] +$ `peso`$(u, v) <$ `distancia`$[v]$, actualizar y registrar predecesor.

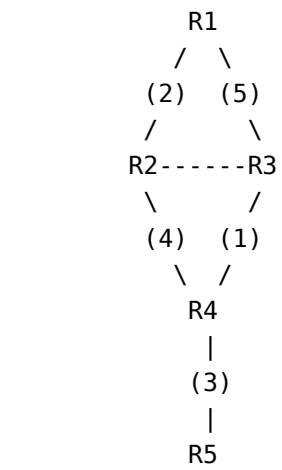
- **Validar** que no existan pesos negativos (Dijkstra no los admite).

6. Interfaz del Programa (Menú)

```
===== SIMULADOR DE ENRUTAMIENTO - DIJKSTRA =====
1. Cargar topologia desde archivo
2. Cargar topologia de ejemplo (hardcodeada)
3. Agregar enlace manualmente
4. Mostrar topologia actual
5. Calcular tabla de rutas desde un router
6. Simular envío de paquete
7. Ejecutar pruebas automáticas
0. Salir
```

Nota: Los puntos 2 y 3 son opcionales.

6.1. Topología de Prueba (ejemplo)



Costos: R1–R2=2, R1–R3=5, R2–R3=1, R2–R4=4, R3–R4=1, R4–R5=3.

Tabla esperada desde R1: R2→2 (R2), R3→3 (R2), R4→4 (R2), R5→7 (R2).

7. Formato del Archivo de Topología

```
# Comentario
ROUTERS 5
NOMBRE 0 R1
NOMBRE 1 R2
NOMBRE 2 R3
NOMBRE 3 R4
NOMBRE 4 R5
ENLACE 0 1 2
ENLACE 0 2 5
ENLACE 1 2 1
ENLACE 1 3 4
```

```
ENLACE 2 3 1
ENLACE 3 4 3
```

8. Criterios de Evaluación

Criterio	**Puntaje**
Correcta implementación del algoritmo de Dijkstra	30 %
Construcción correcta de la tabla de enrutamiento (*next hop*)	15 %
Simulación de envío de paquetes con ruta completa	10 %
Modularización y organización del código	15 %
Manejo de errores y validaciones	10 %
Interfaz de usuario clara y funcional	10 %
Documentación (comentarios + README)	5 %
Extras: min-heap, lista de adyacencia, Makefile, topología dinámica	+10 %

9. Casos de Prueba Obligatorios

El proyecto incluye un módulo `tests.c` que debe ejecutarse desde el menú (opción 7) o con `make test`. Las pruebas se consideran **aprobadas** cuando todas reportan `PASS` y el resumen final muestra `0 fallos`.

Los casos cubiertos son:

1. **Red lineal:** R1–R2–R3–R4 → verificar caminos únicos y suma de costos.
2. **Red con múltiples caminos:** verificar que se elija el de menor costo y que el *next hop* sea correcto.
3. **Red desconectada:** si un destino es inalcanzable, mostrar `"Sin ruta"` y devolver error.
4. **Origen igual a destino:** costo 0, ruta trivial.
5. **Red con ciclo:** verificar que no se generen bucles en la ruta.
6. **Nodo inexistente:** validar que las funciones rechacen parámetros inválidos sin colgarse.
7. **Topología del enunciado:** verificar la tabla de rutas completa desde R1 (*next hop* = R2 para todos los destinos).
8. **Simetría:** el costo de A→B debe coincidir con B→A en un grafo no dirigido.
9. **Eliminación de enlace:** verificar que la topología y las rutas se actualicen correctamente.

Además, se espera que el alumno **agregue al menos 2 pruebas propias** que cubran casos no contemplados (por ejemplo: red completamente desconectada, topología con un solo router, costos muy grandes, etc.).

10. Entregables

Código fuente

- `dijkstra.c`
- `dijkstra.h`
- `grafo.c`
- `grafo.h`
- `enrutamiento.c`
- `enrutamiento.h`
- `main.c` (demostración de uso y Conjunto de pruebas unitarias)
- `Makefile` con targets: `all`, `clean`, `test`, `valgrind`

Estilo

- Indentación consistente (2 o 4 espacios).
- Nombres descriptivos.
- Comentarios en funciones no triviales.

Informe breve (máx. 3 páginas)

- Decisiones de diseño.
- Complejidad temporal de cada operación.
- Dificultades encontradas y cómo se resolvieron.
- Casos de pruebas y resultados.

Declaración de uso de IA

- Indicar si se usó IA generativa, en qué partes, y cómo se verificó el código generado.
- El uso no declarado se considera plagio.

Presentación

El proyecto debe ser entregado **una semana** antes del fin del cuatrimestre.

11. Pistas y Sugerencias

- Comenzar por validar el algoritmo con una topología pequeña antes de agregar funcionalidades.
- Para reconstruir la ruta de un paquete, seguir los predecesores desde el destino hacia atrás y luego invertir.
- **Para el cálculo del *next hop*:** desde el destino, retroceder por los predecesores hasta llegar al nodo cuyo predecesor es el origen; ese nodo es el siguiente salto.
- Compilar con `valgrind` si se usa memoria dinámica.
- Documentar cada función con comentarios estilo Doxygen.

- Ejecutar `make test` con frecuencia: es la forma más rápida de detectar regresiones.

12. Cronograma Sugerido

Semana	Actividad
1	Estudio del marco teórico, diseño de estructuras, carga de topología, menú básico
2	Implementación de Dijkstra, tabla de rutas y simulación de paquetes
3	Pruebas, corrección de errores, documentación e informe

13. Referencias Sugeridas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. — *Introduction to Algorithms* (Cap. 24: Single-Source Shortest Paths).
- Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1, 269–271.
- RFC 2328 — *OSPF Version 2* (sección sobre cálculo del árbol SPF).
- Kernighan, B. W., Ritchie, D. M. — *The C Programming Language* (capítulos de estructuras y punteros).

14. Material Adjunto

Junto con este enunciado se entregan:

- **Esqueleto de código inicial** con estructuras, prototipos.
- **Batería de pruebas automáticas** (`tests.c` / `tests.h`) lista para integrar en el `Makefile`.
- **main.c**
- **Topología de ejemplo** (`topologia.txt`).

El alumno debe completar las funciones, hacer que todas las pruebas pasen y entregar el proyecto con la documentación solicitada.

Nota importante

Restricciones técnicas

- Compilador: `gcc -Wall -Wextra -Werror -std=c11 -g -O0`.
- Memoria: el código debe pasar `valgrind --leak-check=full --error-exitcode=1`.
- El código debe compilar sin warnings con las flags indicadas. Se recomienda compilar localmente con las mismas flags antes de entregar.

Penalizaciones:

- No compila con las flags especificadas: el trabajo se considera desaprobado.
- Memory leaks (valgrind falla): -15 puntos.
- Uso de IA no declarado: se considera plagio.

A. Anexo 1: grafo.h

```c
#ifndef GRAFO_H
#define GRAFO_H

#define MAX_ROUTERS 20
#define INF 999999
#define MAX_NOMBRE  32
#define MAX_LINEA   256

/* Red simulada: matriz de adyacencia + nombres de routers.
 * Convención: matriz[i][j] == INF significa "sin enlace directo".
 *             matriz[i][i] == 0 siempre. */
typedef struct {
    int  numRouters;
    int  matriz[MAX_ROUTERS][MAX_ROUTERS];
    char nombres[MAX_ROUTERS][MAX_NOMBRE];
} Red;

int  inicializarRed(Red *red, int n);
int  setNombreRouter(Red *red, int idx, const char *nombre);
int  agregarEnlace(Red *red, int i, int j, int costo);
int  eliminarEnlace(Red *red, int i, int j);
int  cargarTopologiaDesdeArchivo(Red *red, const char *ruta);
int  cargarTopologiaEjemplo(Red *red);
void mostrarTopologia(const Red *red);
void liberarRed(Red *red);

#endif /* GRAFO_H */
```

B. Anexo 2: dijkstra.h

```c
#ifndef DIJKSTRA_H
#define DIJKSTRA_H

#include "grafo.h"

typedef struct {
    int distancia[MAX_ROUTERS];
    int predecesor[MAX_ROUTERS];
    int visitado[MAX_ROUTERS];
} ResultadoDijkstra;

int dijkstra(const Red *red, int origen, ResultadoDijkstra *res);

#endif /* DIJKSTRA_H */
```

C. Anexo 3: enrutamiento.h

```c
#ifndef ENRUTAMIENTO_H
#define ENRUTAMIENTO_H

#include "dijkstra.h"

typedef struct {
    int destino;
    int costo;
```

```
    int siguienteSalto;   /* -1 si inalcanzable */
} EntradaRuta;

int  construirTablaRutas(const Red *red, int origen,
                         const ResultadoDijkstra *res,
                         EntradaRuta *tabla);
void mostrarTablaRutas(const Red *red, const EntradaRuta *tabla);
int  simularEnvioPaquete(const Red *red, int origen, int destino,
                         const ResultadoDijkstra *res);

#endif /* ENRUTAMIENTO_H */
```

D. Anexo 4: Test automatizados

```
#ifndef TESTS_H
#define TESTS_H

int ejecutarTodasLasPruebas(void);

#endif /* TESTS_H */
```

```
#include <stdio.h>
#include "tests.h"
#include "dijkstra.h"
#include "enrutamiento.h"

static int g_total  = 0;
static int g_fallos = 0;

static void reportar(const char *nombre, int ok, const char *detalle) {
    g_total++;
    if (ok) {
        printf("  [PASS] %-55s\n", nombre);
    } else {
        g_fallos++;
        printf("  [FAIL] %-55s  -> %s\n", nombre, detalle ? detalle : "");
    }
}

static int eq(int a, int b) { return a == b; }

/* ---------------- Caso 1: red lineal ---------------- */
static void test_red_lineal(void) {
    printf("\n--- Caso 1: Red lineal ---\n");
    Red red;
    inicializarRed(&red, 4);
    agregarEnlace(&red, 0, 1, 3);
    agregarEnlace(&red, 1, 2, 2);
    agregarEnlace(&red, 2, 3, 4);

    ResultadoDijkstra res;
    int ok = (dijkstra(&red, 0, &res) == 0);

    reportar("Dijkstra retorna 0", ok, "retorno != 0");
    reportar("R1->R1 = 0", eq(res.distancia[0], 0), "esperado 0");
    reportar("R1->R2 = 3", eq(res.distancia[1], 3), "esperado 3");
    reportar("R1->R3 = 5", eq(res.distancia[2], 5), "esperado 5");
    reportar("R1->R4 = 9", eq(res.distancia[3], 9), "esperado 9");
    reportar("Pred R2 = R1", eq(res.predecesor[1], 0), "esperado 0");
    reportar("Pred R3 = R2", eq(res.predecesor[2], 1), "esperado 1");
```

```
    reportar("Pred R4 = R3", eq(res.predecesor[3], 2), "esperado 2");
}

/* ---------------- Caso 2: múltiples caminos ---------------- */
static void test_multiples_caminos(void) {
    printf("\n--- Caso 2: Múltiples caminos ---\n");
    Red red;
    inicializarRed(&red, 3);
    agregarEnlace(&red, 0, 1, 2);
    agregarEnlace(&red, 0, 2, 5);
    agregarEnlace(&red, 1, 2, 1);

    ResultadoDijkstra res;
    dijkstra(&red, 0, &res);

    reportar("R1->R3 = 3 (vía R2)", eq(res.distancia[2], 3), "esperado 3");
    reportar("Pred R3 = R2", eq(res.predecesor[2], 1), "esperado 1");

    EntradaRuta tabla[MAX_ROUTERS];
    construirTablaRutas(&red, 0, &res, tabla);
    reportar("Next hop R3 = R2", eq(tabla[2].siguienteSalto, 1), "esperado 1");
    reportar("Costo R3 en tabla = 3", eq(tabla[2].costo, 3), "esperado 3");
}

/* ---------------- Caso 3: red desconectada ---------------- */
static void test_red_desconectada(void) {
    printf("\n--- Caso 3: Red desconectada ---\n");
    Red red;
    inicializarRed(&red, 5);
    agregarEnlace(&red, 0, 1, 1);
    agregarEnlace(&red, 1, 2, 1);
    agregarEnlace(&red, 3, 4, 1);

    ResultadoDijkstra res;
    dijkstra(&red, 0, &res);

    reportar("R1->R4 = INF", eq(res.distancia[3], INF), "esperado INF");
    reportar("R1->R5 = INF", eq(res.distancia[4], INF), "esperado INF");
    reportar("Pred R4 = -1", eq(res.predecesor[3], -1), "esperado -1");
    reportar("simular R1->R5 = -1",
             eq(simularEnvioPaquete(&red, 0, 4, &res), -1), "esperado -1");
}

/* ---------------- Caso 4: origen == destino ---------------- */
static void test_origen_igual_destino(void) {
    printf("\n--- Caso 4: Origen == Destino ---\n");
    Red red;
    inicializarRed(&red, 3);
    agregarEnlace(&red, 0, 1, 7);
    agregarEnlace(&red, 1, 2, 7);

    ResultadoDijkstra res;
    dijkstra(&red, 1, &res);
    reportar("R2->R2 = 0", eq(res.distancia[1], 0), "esperado 0");
    reportar("Pred R2 = -1", eq(res.predecesor[1], -1), "esperado -1");
    reportar("simular R2->R2 = 0",
             eq(simularEnvioPaquete(&red, 1, 1, &res), 0), "esperado 0");
}

/* ---------------- Caso 5: red con ciclo ---------------- */
static void test_red_con_ciclo(void) {
    printf("\n--- Caso 5: Red con ciclo ---\n");
    Red red;
```

```
103     inicializarRed(&red, 4);
104     agregarEnlace(&red, 0, 1, 1);
105     agregarEnlace(&red, 1, 3, 1);
106     agregarEnlace(&red, 3, 2, 1);
107     agregarEnlace(&red, 2, 0, 1);
108 
109     ResultadoDijkstra res;
110     dijkstra(&red, 0, &res);
111     reportar("R1->R2 = 1", eq(res.distancia[1], 1), "esperado 1");
112     reportar("R1->R3 = 1", eq(res.distancia[2], 1), "esperado 1");
113     reportar("R1->R4 = 2", eq(res.distancia[3], 2), "esperado 2");
114     reportar("simular R1->R4 OK",
115             eq(simularEnvioPaquete(&red, 0, 3, &res), 0), "esperado 0");
116 }
117 
118 /* ---------------- Caso 6: nodo inexistente / validaciones ---------------- */
119 static void test_nodo_inexistente(void) {
120     printf("\n--- Caso 6: Nodo inexistente y validaciones ---\n");
121     Red red;
122     inicializarRed(&red, 3);
123     agregarEnlace(&red, 0, 1, 1);
124 
125     ResultadoDijkstra res;
126     reportar("dijkstra(origen=-1) = -1", eq(dijkstra(&red, -1, &res), -1), "esperado -1");
127     reportar("dijkstra(origen=99) = -1", eq(dijkstra(&red, 99, &res), -1), "esperado -1");
128     reportar("dijkstra(res=NULL) = -1",  eq(dijkstra(&red, 0, NULL), -1), "esperado -1");
129 
130     reportar("agregarEnlace(i=-1) = -1",
131             eq(agregarEnlace(&red, -1, 1, 5), -1), "esperado -1");
132     reportar("agregarEnlace(costo=0) = -1",
133             eq(agregarEnlace(&red, 0, 2, 0), -1), "esperado -1");
134     reportar("agregarEnlace(costo<0) = -1",
135             eq(agregarEnlace(&red, 0, 2, -3), -1), "esperado -1");
136     reportar("agregarEnlace(i==j) = -1",
137             eq(agregarEnlace(&red, 1, 1, 5), -1), "esperado -1");
138     reportar("agregarEnlace duplicado = -1",
139             eq(agregarEnlace(&red, 0, 1, 9), -1), "esperado -1");
140 }
141 
142 /* ---------------- Extra 1: topología del enunciado ---------------- */
143 static void test_topologia_enunciado(void) {
144     printf("\n--- Extra 1: Topología del enunciado ---\n");
145     Red red;
146     cargarTopologiaEjemplo(&red);
147 
148     ResultadoDijkstra res;
149     dijkstra(&red, 0, &res);
150     reportar("R1->R2 = 2", eq(res.distancia[1], 2), "esperado 2");
151     reportar("R1->R3 = 3", eq(res.distancia[2], 3), "esperado 3");
152     reportar("R1->R4 = 4", eq(res.distancia[3], 4), "esperado 4");
153     reportar("R1->R5 = 7", eq(res.distancia[4], 7), "esperado 7");
154 
155     EntradaRuta tabla[MAX_ROUTERS];
156     construirTablaRutas(&red, 0, &res, tabla);
157     reportar("Next hop R2 = R2", eq(tabla[1].siguienteSalto, 1), "esperado 1");
158     reportar("Next hop R3 = R2", eq(tabla[2].siguienteSalto, 1), "esperado 1");
159     reportar("Next hop R4 = R2", eq(tabla[3].siguienteSalto, 1), "esperado 1");
160     reportar("Next hop R5 = R2", eq(tabla[4].siguienteSalto, 1), "esperado 1");
161 }
162 
163 /* ---------------- Extra 2: simetría no dirigida ---------------- */
164 static void test_simetria(void) {
165     printf("\n--- Extra 2: Simetría ---\n");
```

```
166     Red red;
167     inicializarRed(&red, 4);
168     agregarEnlace(&red, 0, 1, 4);
169     agregarEnlace(&red, 1, 2, 3);
170     agregarEnlace(&red, 2, 3, 2);
171     agregarEnlace(&red, 0, 3, 20);
172 
173     ResultadoDijkstra a, b;
174     dijkstra(&red, 0, &a);
175     dijkstra(&red, 3, &b);
176 
177     reportar("R1->R4 == R4->R1",
178             eq(a.distancia[3], b.distancia[0]), "distintos");
179     reportar("R1->R3 == R3->R1",
180             eq(a.distancia[2], b.distancia[2]), "distintos");
181     reportar("R1->R4 = 9", eq(a.distancia[3], 9), "esperado 9");
182 }
183 
184 /* ---------------- Extra 3: eliminar enlace ---------------- */
185 static void test_eliminar_enlace(void) {
186     printf("\n--- Extra 3: Eliminación de enlace ---\n");
187     Red red;
188     inicializarRed(&red, 3);
189     agregarEnlace(&red, 0, 1, 1);
190     agregarEnlace(&red, 1, 2, 1);
191     agregarEnlace(&red, 0, 2, 10);
192 
193     ResultadoDijkstra res;
194     dijkstra(&red, 0, &res);
195     reportar("Antes R1->R3 = 2", eq(res.distancia[2], 2), "esperado 2");
196 
197     reportar("eliminar R1-R2 = 0",
198             eq(eliminarEnlace(&red, 0, 1), 0), "esperado 0");
199 
200     dijkstra(&red, 0, &res);
201     reportar("Después R1->R3 = 10", eq(res.distancia[2], 10), "esperado 10");
202     reportar("eliminar inexistente = -1",
203             eq(eliminarEnlace(&red, 0, 1), -1), "esperado -1");
204 }
205 
206 /* ---------------- Punto de entrada ---------------- */
207 int ejecutarTodasLasPruebas(void) {
208     g_total = 0; g_fallos = 0;
209 
210     printf("\n==========================================\n");
211     printf("      BATERÍA DE PRUEBAS AUTOMÁTICAS\n");
212     printf("==========================================\n");
213 
214     test_red_lineal();
215     test_multiples_caminos();
216     test_red_desconectada();
217     test_origen_igual_destino();
218     test_red_con_ciclo();
219     test_nodo_inexistente();
220     test_topologia_enunciado();
221     test_simetria();
222     test_eliminar_enlace();
223 
224     printf("\n==========================================\n");
225     printf("  RESUMEN: %d pruebas, %d fallos\n", g_total, g_fallos);
226     printf("  RESULTADO: %s\n",
227            (g_fallos == 0) ? "TODAS LAS PRUEBAS PASARON" : "HAY FALLOS");
228     printf("==========================================\n\n");
```

```
    return g_fallos;
}
```

E. Anexo 5: main.c

```c
#include <stdio.h>
#include <string.h>
#include "grafo.h"
#include "dijkstra.h"
#include "enrutamiento.h"
#include "tests.h"

static void limpiarBuffer(void) {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) { /* descartar */ }
}

static int leerEntero(const char *msg, int *out) {
    printf("%s", msg);
    if (scanf("%d", out) != 1) { limpiarBuffer(); return -1; }
    limpiarBuffer();
    return 0;
}

static void mostrarMenu(void) {
    printf("\n===== SIMULADOR DE ENRUTAMIENTO - DIJKSTRA =====\n");
    printf("1. Cargar topologia desde archivo\n");
    printf("2. Cargar topologia de ejemplo\n");
    printf("3. Agregar enlace manualmente\n");
    printf("4. Mostrar topologia actual\n");
    printf("5. Calcular tabla de rutas desde un router\n");
    printf("6. Simular envio de paquete\n");
    printf("7. Ejecutar pruebas automaticas\n");
    printf("0. Salir\n");
    printf("Opcion: ");
}

static void opcionCargarArchivo(Red *red) {
    char ruta[MAX_LINEA];
    printf("Ruta del archivo: ");
    if (scanf("%255s", ruta) != 1) { limpiarBuffer(); return; }
    limpiarBuffer();

    if (cargarTopologiaDesdeArchivo(red, ruta) == 0)
        printf("Topologia cargada desde '%s'.\n", ruta);
    else
        printf("No se pudo cargar la topologia.\n");
}

static void opcionAgregarEnlace(Red *red) {
    if (red->numRouters == 0) { printf("Primero cargue una topologia.\n"); return; }
    int i, j, c;
    if (leerEntero("Indice router 1: ", &i) != 0) return;
    if (leerEntero("Indice router 2: ", &j) != 0) return;
    if (leerEntero("Costo: ", &c) != 0) return;

    if (agregarEnlace(red, i, j, c) == 0)
        printf("Enlace %d-%d agregado (costo %d).\n", i, j, c);
    else
        printf("Error: indices invalidos, costo <= 0 o enlace ya existente.\n");
}
```

```c

static void opcionTablaRutas(const Red *red) {
    if (red->numRouters == 0) { printf("Primero cargue una topologia.\n"); return; }
    int origen;
    if (leerEntero("Indice del router origen: ", &origen) != 0) return;
    if (origen < 0 || origen >= red->numRouters) {
        printf("Indice fuera de rango (0..%d).\n", red->numRouters - 1);
        return;
    }

    ResultadoDijkstra res;
    if (dijkstra(red, origen, &res) != 0) { printf("Error en Dijkstra.\n"); return; }

    EntradaRuta tabla[MAX_ROUTERS];
    construirTablaRutas(red, origen, &res, tabla);

    printf("\nTabla de rutas desde %s:\n", red->nombres[origen]);
    mostrarTablaRutas(red, tabla);
}

static void opcionSimularPaquete(const Red *red) {
    if (red->numRouters == 0) { printf("Primero cargue una topologia.\n"); return; }
    int o, d;
    if (leerEntero("Router origen: ", &o) != 0) return;
    if (leerEntero("Router destino: ", &d) != 0) return;
    if (o < 0 || o >= red->numRouters || d < 0 || d >= red->numRouters) {
        printf("Indices fuera de rango.\n");
        return;
    }

    ResultadoDijkstra res;
    dijkstra(red, o, &res);
    simularEnvioPaquete(red, o, d, &res);
}

int main(int argc, char *argv[]) {
    /* Modo no interactivo: solo pruebas (útil para CI / make test) */
    if (argc > 1 && strcmp(argv[1], "--test") == 0)
        return ejecutarTodasLasPruebas();

    Red red;
    inicializarRed(&red, 0);

    int opcion;
    do {
        mostrarMenu();
        if (scanf("%d", &opcion) != 1) { limpiarBuffer(); continue; }
        limpiarBuffer();

        switch (opcion) {
            case 1: opcionCargarArchivo(&red); break;
            case 2:
                cargarTopologiaEjemplo(&red);
                printf("Topologia de ejemplo cargada (5 routers).\n");
                break;
            case 3: opcionAgregarEnlace(&red); break;
            case 4: mostrarTopologia(&red);    break;
            case 5: opcionTablaRutas(&red);    break;
            case 6: opcionSimularPaquete(&red); break;
            case 7: {
                int fallos = ejecutarTodasLasPruebas();
                printf("Pruebas fallidas: %d\n", fallos);
                break;
```

```
            }
            case 0: printf("Saliendo...\n"); break;
            default: printf("Opcion invalida.\n"); break;
        }
    } while (opcion != 0);

    liberarRed(&red);
    return 0;
}
```

F. Anexo 6: Makefile de ejemplo

```
CC      = gcc
CFLAGS  = -std=c99 -Wall -Wextra -Wpedantic -g -O0
TARGET  = simulador_rutas
OBJS    = main.o grafo.o dijkstra.o enrutamiento.o tests.o

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

run: all
    ./$(TARGET)

test: all
    ./$(TARGET) --test

clean:
    rm -f $(OBJS) $(TARGET)

valgrind: $(TARGET)
    valgrind --leak-check=full --error-exitcode=1 ./$(TARGET)

.PHONY: all run test clean
```

Listing 1: Makefile de ejemplo.

G. Anexo 7: Ejemplo de bash de ejecución

```
make        # compila
make run    # ejecuta
make valgrind     # verifica leaks
# make clean  # limpia
```

Listing 2: Archivo bash de ejecución.

Referencias

[1] J. Moy, *OSPF Version 2*, STD 54, RFC 2328, Internet Engineering Task Force, abril de 1998. DOI: 10.17487/RFC2328.

[2] R. Coltun, D. Ferguson, J. Moy y A. Lindem, *OSPF for IPv6*, RFC 5340, Internet Engineering Task Force, julio de 2008. DOI: 10.17487/RFC5340.

[3] G. Malkin, *RIP Version 2*, STD 56, RFC 2453, Internet Engineering Task Force, noviembre de 1998. DOI: 10.17487/RFC2453.

[4] J. T. Moy, *OSPF: Anatomy of an Internet Routing Protocol*, Addison-Wesley, 1998.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3.ª ed., MIT Press, 2009. (Cap. 24: *Single-Source Shortest Paths*).

[6] J. F. Kurose y K. W. Ross, *Computer Networking: A Top-Down Approach*, 7.ª ed., Pearson, 2017.

[7] A. S. Tanenbaum y D. J. Wetherall, *Computer Networks*, 5.ª ed., Pearson, 2011.